

Critical Analysis and Reflection: CompanyMan App

Framework Overview

For the development of **CompanyManMobile**, I adopted a cross-platform approach utilising **React Native** (via the **Expo Managed Workflow**) for the frontend, paired with a **Node.js** and **Express** backend connected to a **MongoDB** database.

This framework selection was driven by the "Write Once, Run Anywhere" paradigm, allowing for the simultaneous deployment of iOS and Android applications from a single JavaScript/TypeScript codebase. The **Expo Managed Workflow** was specifically used to abstract the complexity of native build configurations, enabling rapid iteration on hardware features, such as the camera and filesystem, without requiring deep proficiency in Swift or Kotlin. Furthermore, adopting the **MERN-stack** architecture provided language consistency across the full stack, streamlining data validation and interface definitions between the client and the API.

Advantages and Limitations

The primary advantage of this framework was development velocity. The unified stack allowed for the rapid construction of the MVP, including the secure login system and backend connection. The use of Expo Router facilitated a file-based routing system that mirrored web development structures, significantly reducing the boilerplate code typically required for navigation. Additionally, the ecosystem of pre-optimised libraries, such as react-native-paper for Material Design UI and expo-secure-store for JWT handling, accelerated the implementation of features like inventory modals and secure authentication.

However, the framework imposed limitations regarding native module integration and performance. Relying on the JavaScript bridge for computationally heavy tasks introduced potential performance bottlenecks compared to fully native solutions. This was most evident during real-time barcode processing, where the asynchronous communication between the native camera module and the JS thread caused latency, although enabling the **Hermes engine** helped mitigate this by optimising the JavaScript bundle execution.

Implementation and Challenges

The implementation focused on creating a robust internal tool for restaurant management. The architecture relied heavily on the React Context API (specifically OrderContext) to manage global states like the POS shopping cart, while the backend utilised custom middleware for Role-Based Access Control (RBAC).

Key Challenges and Resolutions

1. **Hardware Event Race Conditions:** A significant challenge arose when integrating the barcode scanner. The hardware fired the onScanned event multiple times per second, faster than the React render cycle could update the state, causing the "Add Item" modal to trigger repeatedly and freeze the UI.

Resolution: I implemented a useRef locking mechanism. Unlike useState, which relies on the asynchronous render cycle, a React Ref updates instantly. By checking scanGuardRef.current before processing a scan, I enforced a strict "single-scan" behaviour, preventing duplicate data processing.

2. **Asynchronous Storage & Auth State:** A "401 Token" error occurred during the login flow because the application attempted to fetch dashboard data before the JWT was fully persisted to the device's secure storage.

Resolution: This was addressed by implementing a "Loading Guard" pattern within the custom AuthContext. This ensures the application waits for the storage promise to resolve and confirm authentication before attempting any protected API calls.

3. **Complex State Synchronisation:** The Staff Scheduler required the view to automatically scroll to the current day upon opening. Initially, this caused conflicts where the list attempted to scroll before the large dataset was fetched and rendered, leading to layout crashes.

Resolution: I architected a two-stage hook system: a useFocusEffect to reset the calendar state, and a separate useEffect listening for the data loading state to trigger the scroll action only once the list was fully hydrated.

Reflection on Effectiveness

React Native proved highly effective for the project's specific needs, particularly in enabling the **Internal POS** system. The framework's ability to handle atomic transactions ensured that revenue logging and inventory deduction happened simultaneously, preserving data integrity. The integration with **Google Gemini API** also demonstrated the framework's versatility in handling external API responses and rendering them as natural-language "Daily Briefs" within a native mobile interface.

Reflecting on the development, the initial approach to the Staff Scheduler (an auto-scrolling list) was flawed due to list virtualisation rigidity. If starting again, I would bypass the custom list architecture entirely in favor of the stable monthly view implementation used in the final build. Additionally, I would prioritize an "Offline First" architecture earlier in the lifecycle. Retrofitting offline capabilities into a cloud-dependent React Native app proved significantly harder than building them from the outset.

Future Considerations

Future development should focus on enhancing financial analytics. Currently, the app calculates Gross Profit; however, leveraging the Scheduler data (Hours × Hourly Rate) would allow for the calculation of Labor Costs. By integrating this with fixed inputs for utilities, the app could provide a "Net Profit" estimation, adding significant business value.

To address the dependency on constant internet connectivity, integrating a local database engine like **SQLite** or **WatermelonDB** would be beneficial. This would enable a robust "Offline Mode," allowing waitstaff to take orders and view menus during internet outages, with data syncing to MongoDB once connectivity is restored. This hybrid approach would maximise the utility of the mobile framework in real-world, fast-paced hospitality environments.

Reflection on Effectiveness

The React Native framework was highly effective for this data-driven application. The use of React Context (OrderContext) effectively managed the global state for the POS system, allowing the "Cart" data to persist seamlessly between the Menu tab and the Checkout modal.

However, if starting again, I would make two key changes:

1. **State Management:** While Context API worked, scaling to include complex features like "Shift Swapping" or "Real-time Kitchen Updates" would justify using a dedicated state library like Zustand or Redux Toolkit to avoid unnecessary re-renders in the main _layout.tsx.
2. **Strict Typing:** I would enforce strict TypeScript definitions from day one. I encountered several bugs (e.g., toFixed crashing on undefined prices) that could have been prevented with stricter

interfaces for the API responses.

Future Considerations

Future development should focus on enhancing financial analytics. Currently, the app calculates Gross Profit; however, leveraging the Scheduler data (Hours × Hourly Rate) would allow for the calculation of Labor Costs. By integrating this with fixed inputs for utilities, the app could offer a "Net Profit" estimation, adding significant business value.

To address the dependency on constant internet connectivity, integrating a local database engine like **SQLite** or **WatermelonDB** would be beneficial. This would enable a robust "Offline Mode," allowing waitstaff to take orders and view menus during internet outages, with data syncing to MongoDB once connectivity is restored. This hybrid approach would maximise the utility of the mobile framework in real-world, fast-paced hospitality environments.